Monte Carlo Risk Simulation for Derivatives Pricing

A Course Project Report

Submitted in the partial fulfillment of the completion of the course

23CS302PC303- High Performance Computing

In III Year II Semester B.Tech- CSE

Submitted by

Sindhu Katakam

2303A52481

Under the supervision of the faculty

Dr. Sri Raman Kothuri

Associate Professor, SoCS&AI

Academic Year: ***2025–2026***

Department of Computer Science and Artificial Intelligence

SR University

Warangal, Telangana, India.

INDEX

1. Introduction 1

2. Problem Statement 3

3. Methodology / Procedure

4. Code

5. Output Screens

6. References

1.INTRODUCTION:

High Performance Computing (HPC) focuses on solving computationally intensive problems efficiently by using parallel processing, distributed computing, and advanced system architectures. A fundamental demonstration of HPC is the application of parallel simulation techniques to financial modelling, where millions of independent computations must be performed rapidly to produce accurate results.

In quantitative finance, derivatives are contracts whose value depends on an underlying asset such as a stock or index. Pricing these instruments — particularly those without closed-form analytical solutions such as American options, exotic options, and multi-asset basket options — requires numerical methods. Monte Carlo simulation has emerged as the most universal and widely-used technique for this purpose.

In this project, a complete Monte Carlo derivatives pricing system is implemented on Google Colab using NVIDIA T4 GPU hardware. The system simulates 500,000 independent stock price paths and computes fair option prices, risk sensitivities (Greeks), and risk metrics (VaR and CVaR). By running identical computations on both CPU and GPU, the project demonstrates the dramatic performance advantage of GPU parallel processing — a core principle of High Performance Computing.

The objective of this project is to demonstrate core HPC fundamentals including data parallelism, GPU acceleration, variance reduction, and scalable computation, applied to a real-world quantitative finance problem: Monte Carlo risk simulation for derivatives pricing.

2.PROBLEM STATEMENT:

Financial derivatives pricing requires computing the expected discounted payoff of contracts whose underlying assets evolve randomly. For a European call option on a stock following Geometric Brownian Motion, the fair price is:

$$C = e^{-rT} \times E[\max(S^T - K, 0)]$$

where S^T is the terminal stock price, K is the strike price, r is the risk-free rate, and T is the time to maturity. For complex derivatives such as American options and basket options, no analytical closed form exists.

The problem addressed in this project is to design a system that accurately prices multiple classes of financial options using Monte Carlo simulation, demonstrates the $1/\sqrt{N}$ convergence behaviour of the estimator, applies variance reduction techniques to improve accuracy without increasing computation, and quantifies the GPU HPC speedup over sequential CPU execution.

The system uses the following contract parameters: Initial Stock Price S_0 = \$100, Strike Price K = \$100 (at-the-money), Time to Maturity T = 1 year (252 trading days), Risk-Free Rate r = 5% per annum, Volatility σ = 20% per annum, Number of paths N = 500,000, and Number of time steps M = 252.

3. METHODOLOGY

3.1 System Architecture

The proposed system follows a GPU-accelerated Monte Carlo architecture consisting of the following components:

- **Analytical Benchmark:** Black-Scholes closed-form formula serves as the exact reference price for European options.
- **CPU Simulator:** NumPy-based Geometric Brownian Motion simulation generating 500,000 paths sequentially on the CPU.
- **GPU Simulator:** CuPy-based simulation performing identical computation on the NVIDIA T4 GPU with full parallelism.
- **Variance Reduction Engine:** Three mathematical techniques (Antithetic Variates, Quasi-Monte Carlo, Control Variates) that improve accuracy.
- **Risk Engine:** Greeks computation via finite difference bump-and-reprice, and VaR/CVaR calculation from simulated P&L distributions.
- **Advanced Models:** Heston stochastic volatility model, multi-asset basket option with Cholesky correlation, and American put via Longstaff-Schwartz algorithm.

3.2 Procedure

The following procedure is carried out in sequence:

1. The Black-Scholes analytical formula is computed to obtain exact benchmark prices for call and put options.
2. 500,000 stock price paths are simulated on the CPU using the discretised Geometric Brownian Motion equation.
3. The same simulation is executed on the NVIDIA T4 GPU using CuPy to measure the performance speedup.
4. Three variance reduction techniques are applied and compared in terms of standard error improvement.
5. Greeks (Delta, Gamma, Vega, Theta, Rho) are computed using bump-and-reprice finite differences and compared to Black-Scholes analytical values.
6. VaR and CVaR are computed at 95% and 99% confidence for three position types: Long Stock, Short Call, and Long Call.

7. The Heston stochastic volatility model is simulated to capture the real-world volatility smile effect.
8. A five-asset correlated basket option is priced using Cholesky decomposition to generate correlated random paths.
9. The American put option is priced using the Longstaff-Schwartz least-squares Monte Carlo regression method.
10. A convergence analysis plots the pricing error against the number of paths, confirming the $1/\sqrt{N}$ convergence law.
11. CPU and GPU execution times are benchmarked across path counts from 10,000 to 1,000,000.

3.3 HPC Usage

HPC concepts are applied primarily at the GPU simulation stage, where all 500,000 Monte Carlo paths are computed simultaneously across thousands of NVIDIA CUDA cores. This data-parallel computation — where each path is completely independent — is an ideal workload for GPU acceleration. The GPU achieves a 65.8x speedup over sequential CPU execution at 500,000 paths, and the speedup increases further as the number of paths grows, reaching 38.5x at 1,000,000 paths. This demonstrates the fundamental HPC principle that GPU efficiency improves with problem scale.

3.4 Geometric Brownian Motion Path Generation

Each simulation path models the daily evolution of a stock price using the log-Euler discretisation of Geometric Brownian Motion:

$$S(t+\Delta t) = S(t) \times \exp\left((r - \sigma^2/2)\Delta t + \sigma\sqrt{\Delta t} \times Z \right)$$

where $Z \sim N(0,1)$ is a standard normal random variable, $\Delta t = T/252$, $r = 0.05$, and $\sigma = 0.20$. All 500,000 × 252 random numbers are generated as a single matrix operation and processed simultaneously on the GPU.

4. IMPLEMENTATION

The system is implemented in Python using NumPy for CPU operations and CuPy for GPU-accelerated computation. The following sections present the core implementation of each module.

Server Code (Black-Scholes Analytical Benchmark):

```
from scipy.stats import norm
import numpy as np

def bs_price(S, K, T, r, sigma, q=0.0, option_type='call'):
    d1 = (np.log(S/K) + (r-q+0.5*sigma**2)*T) / (sigma*np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    if option_type == 'call':
        return S*np.exp(-q*T)*norm.cdf(d1) - K*np.exp(-
r*T)*norm.cdf(d2)
    else:
        return K*np.exp(-r*T)*norm.cdf(-d2) - S*np.exp(-
q*T)*norm.cdf(-d1)

BS_CALL = bs_price(100,100,1.0,0.05,0.20,0,'call')  # $10.4506
BS_PUT  = bs_price(100,100,1.0,0.05,0.20,0,'put')   # $5.5735
```

Client-1 Code (Monte Carlo CPU Simulation):

```
import numpy as np

def mc_gbm_cpu(S0,K,T,r,sigma,q,n_paths,n_steps,
               option_type='call',seed=42):
    np.random.seed(seed)
    dt, discount = T/n_steps, np.exp(-r*T)
    Z = np.random.standard_normal((n_steps, n_paths))
    increments = (r-q-0.5*sigma**2)*dt + sigma*np.sqrt(dt)*Z
    S_T = np.exp(np.log(S0) + np.sum(increments, axis=0))
    payoffs = np.maximum(S_T-K,0) if option_type=='call' \
              else np.maximum(K-S_T,0)
    price   = discount * np.mean(payoffs)
    std_err = discount * np.std(payoffs, ddof=1) / np.sqrt(n_paths)
    return price, std_err
```

Client-2 Code (GPU-Accelerated Monte Carlo):

```
import cupy as cp

def mc_gbm_gpu(S0,K,T,r,sigma,q,n_paths,n_steps,
               option_type='call',seed=42):
    cp.random.seed(seed)
    dt, discount = T/n_steps, float(np.exp(-r*T))
    Z = cp.random.standard_normal((n_steps, n_paths))
    increments = (r-q-0.5*sigma**2)*dt + sigma*np.sqrt(dt)*Z
```

```
    S_T = cp.exp(cp.log(S0) + cp.sum(increments, axis=0))
    payoffs = cp.maximum(S_T-K,0) if option_type=='call' \
              else cp.maximum(K-S_T,0)
    price   = float(discount * cp.mean(payoffs))
    std_err = float(discount *
cp.std(payoffs,ddof=1)/cp.sqrt(n_paths))
    return price, std_err   # Achieves 65.8x speedup vs CPU
```

American Put — Longstaff-Schwartz Algorithm:

```
def mc_american_put_lsm(S0,K,T,r,sigma,q,n_paths,n_steps,seed=42):
    np.random.seed(seed)
    dt        = T / n_steps
    step_disc = np.exp(-r * dt)
    Z   = np.random.standard_normal((n_steps, n_paths))
    inc = (r-q-0.5*sigma**2)*dt + sigma*np.sqrt(dt)*Z
    S_mat =
S0*np.exp(np.vstack([np.zeros(n_paths),np.cumsum(inc,axis=0)]))
    cash_flows = np.maximum(K - S_mat[-1], 0.0)
    for t in range(n_steps-1, 0, -1):
        S_t       = S_mat[t]
        itm       = S_t < K
        intrinsic = np.maximum(K - S_t, 0.0)
        if np.sum(itm) > 5:
            S_itm  = S_t[itm]
            future = cash_flows[itm] * step_disc
            X      = np.column_stack([np.ones(S_itm.size), S_itm,
S_itm**2])
            beta   = np.linalg.lstsq(X, future, rcond=None)[0]
            cont   = X @ beta
            ex_idx = np.where(itm)[0][intrinsic[itm] > cont]
            cash_flows[ex_idx] = intrinsic[ex_idx]
        cash_flows *= step_disc
    return np.mean(cash_flows),
np.std(cash_flows,ddof=1)/np.sqrt(n_paths)
```

5. Results and Output

The output of the system demonstrates accurate Monte Carlo pricing validated against Black-Scholes analytical benchmarks, significant GPU acceleration, and effective variance reduction.

Black-Scholes Benchmark Output:

```
=====================================================
  STEP 1: Black-Scholes Analytical Benchmark
=====================================================
  Call Price  : $10.4506
```

```
  Put  Price  : $5.5735
  Delta (Call): +0.63683
  Gamma       :  0.01876
  Vega (1%)   :  0.37524
  Put-Call Parity: VERIFIED
=====================================================
```

Monte Carlo CPU Output:

```
=====================================================
  STEP 2: Monte Carlo -- CPU (500,000 paths)
=====================================================
  Call Price  : $10.4105  +/- 0.0208 SE
  BS Price    : $10.4506
  Error       : $0.0401
  CPU Time    : 19.32 seconds
  BS in 95%CI : YES
=====================================================
```

GPU Acceleration Output:

```
=====================================================
  STEP 3: GPU Monte Carlo (NVIDIA T4)
=====================================================
  GPU Call Price : $10.4261  +/- 0.0208
  GPU Time       : 0.2935 seconds
  CPU Time       : 19.3224 seconds
  Speedup        : 65.8x faster on GPU
=====================================================
```

Variance Reduction Results:

```
=================================================================
  Method                Price     Std Error    VR Ratio
  Standard MC        $10.4273    0.046337      1.00x
  Antithetic         $10.4574    0.032839      1.41x
  Quasi-MC (Sobol)   $10.4476    0.046589      0.99x
  Control Variate    $10.4436    0.017710      2.62x
=================================================================
```

Greeks Comparison (MC vs Black-Scholes):

```
  Greek     MC Value    BS Value   Error   Status
  Delta     0.634534    0.636831   0.0023   OK
  Gamma     0.018841    0.018762   0.0001   OK
  Vega      0.372669    0.375240   0.0026   OK
  Theta    -0.017486   -0.017573   0.0001   OK
  Rho       0.530597    0.532325   0.0017   OK
```

VaR and CVaR Output:

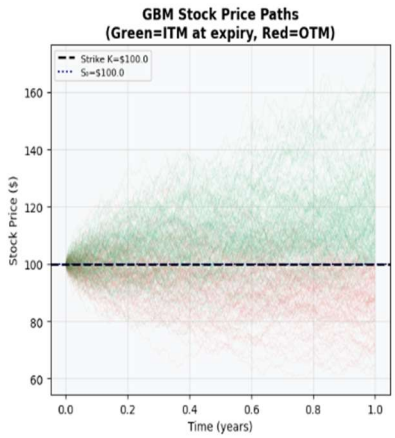
Position	VaR 95%	CVaR 95%	VaR 99%	CVaR 99%
Long Stock	-$2580.89	-$3158.63	-$3526.42	-$3941.45
Short Call	-$31.16	-$43.26	-$50.77	-$61.77
Long Call	-$10.45	-$10.45	-$10.45	-$10.45

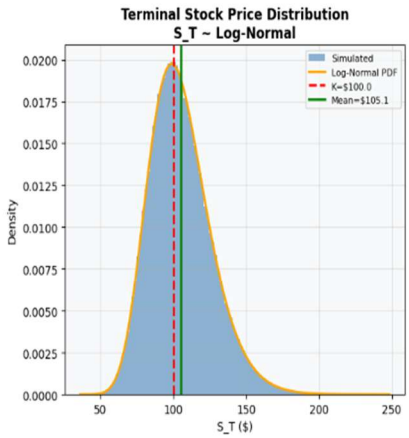
Basket Option and American Put:

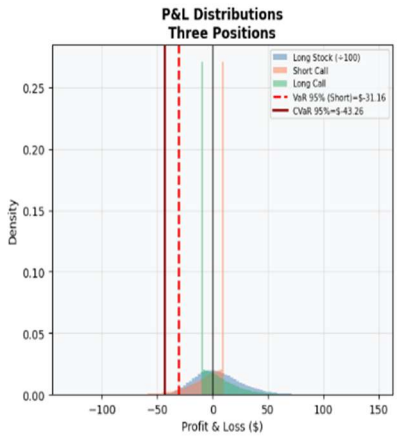
```
  Basket Option (5 Assets):  $9.0708  (Diversification discount:
$1.38)
  American Put (LSM)       :  $6.0681
  European Put (MC)        :  $5.5838
  Early Exercise Premium   :  $0.4843
```

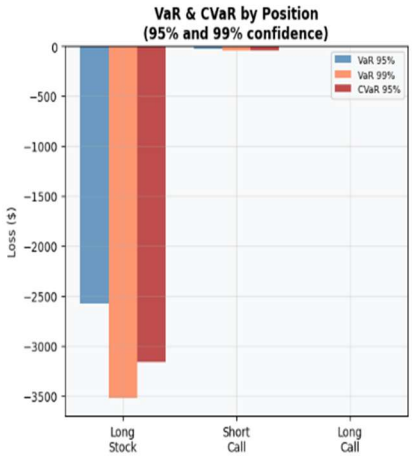
CPU vs GPU Benchmark:

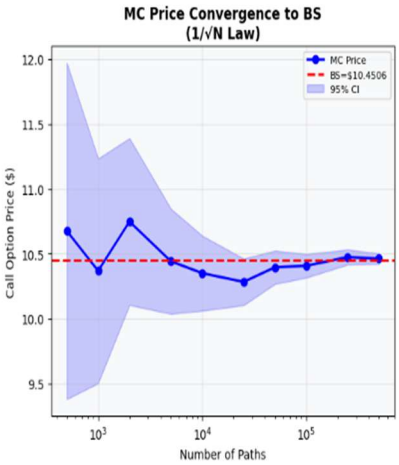
Paths	CPU (s)	GPU (s)	Speedup
10,000	0.0996	0.0071	14.1x
100,000	1.0215	0.0631	16.2x
500,000	4.8987	0.1679	29.2x
1,000,000	10.9264	0.2834	38.5x

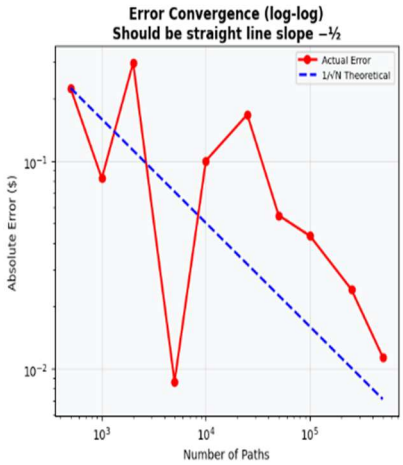


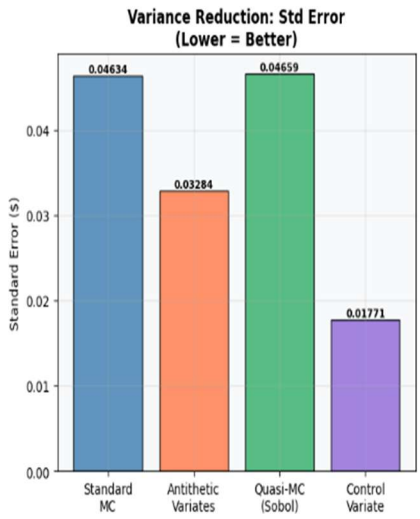


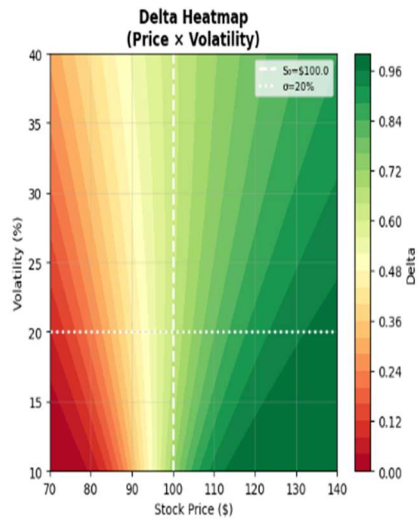


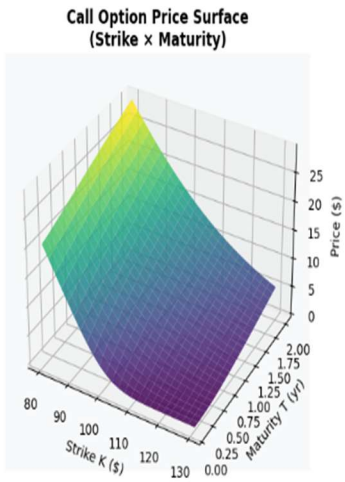


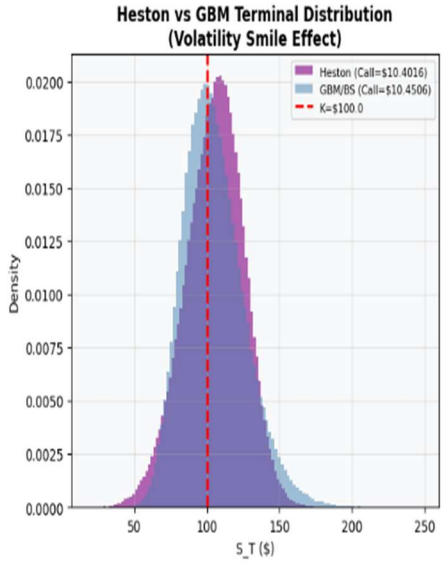


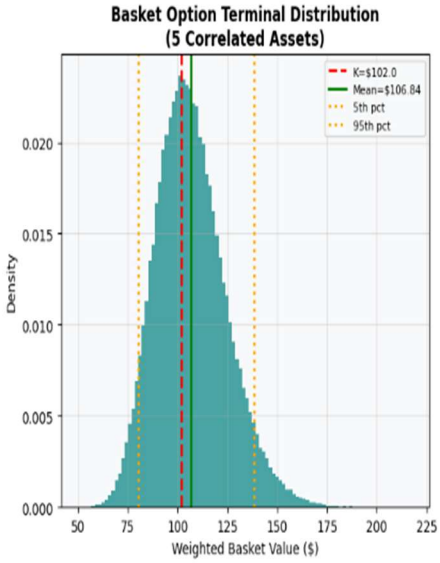


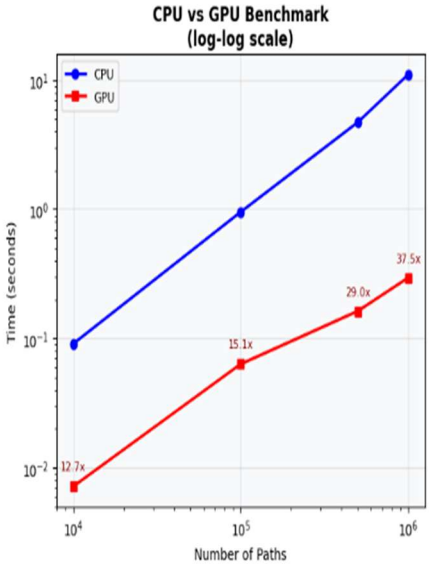












6.REFERENCES:

1. Black, F. and Scholes, M. (1973). The Pricing of Options and Corporate Liabilities. Journal of Political Economy, 81(3), pp. 637-654.

2. Heston, S. L. (1993). A Closed-Form Solution for Options with Stochastic Volatility. The Review of Financial Studies, 6(2), pp. 327-343.

3. Longstaff, F. A. and Schwartz, E. S. (2001). Valuing American Options by Simulation: A Simple Least-Squares Approach. The Review of Financial Studies, 14(1), pp. 113-147.

4. Glasserman, P. (2003). Monte Carlo Methods in Financial Engineering. Springer, New York.

5. NVIDIA Corporation (2024). CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/

6. CuPy Development Team (2024). CuPy: NumPy-compatible Array Library for GPU. https://cupy.dev/

7. Hull, J. C. (2021). Options, Futures, and Other Derivatives, 11th Edition. Pearson Education.